

QVerse — Week 15: Capstone Sprint I — Build a Reproducible Prototype

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `15_capstone_build.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 15; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Turn a narrow question into a quantum experiment.
- Define a hypothesis and success metric before coding.
- Create a minimum working implementation.
- Add sanity tests and a baseline.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. The capstone is an experiment, not just code

A strong beginner capstone should answer a narrow question using a reproducible quantum-computing experiment.

The target is not a large application. The target is a small project where another student can understand:

- the question;
- the quantum concept;
- the circuit;
- the prediction;
- the measured result;
- the validation method;
- the limitations.

A useful project can be technically modest if the experimental reasoning is strong.

2. Start with a research or engineering question

A good question is specific enough to test.

Weak:

How does quantum computing work?

Better:

How does the success probability of a three-qubit Grover search change as the number of Grover iterations increases?

Or:

How does two-qubit depolarizing error affect the undesired-outcome rate of a Bell-state circuit?

A strong question identifies a variable, an observable outcome, or a comparison.

3. Define a hypothesis before coding

The hypothesis states what you expect and why.

Example:

For one marked state among eight possibilities, two Grover iterations should outperform zero or one iteration, while additional iterations eventually reduce success probability because Grover dynamics overshoot the target state.

The prediction should come from theory developed earlier in the course.

Writing it before execution reduces the temptation to invent an explanation after seeing the result.

4. Choose a success metric

The success metric should be quantitative whenever possible.

Examples:

- target-state probability;
- Bell undesired-outcome fraction;
- state fidelity;
- expectation value;
- circuit depth;
- two-qubit gate count;
- absolute error from a theoretical value;
- minimum variational energy;
- decoding success rate.

A project without a metric is difficult to validate.

5. Define a baseline

A baseline gives the result context.

Possible baselines:

- empty or identity circuit;
- zero Grover iterations;
- ideal simulator;
- unoptimized transpilation;
- product state compared with Bell state;
- classical/random guessing;
- exact analytic solution for a toy variational problem.

The baseline should answer:

Better than what?

6. Minimum working circuit

The first implementation should be the smallest circuit that tests the core idea.

Do not begin with every desired feature.

A recommended sequence is:

1. make one small ideal example work;
2. validate its expected state or distribution;
3. add one automated sanity check;

4. only then extend the experiment.

This reduces debugging complexity.

7. Sanity tests

A sanity test checks behavior that should be obvious.

Examples:

- $H^2 = I$;
- QFT followed by inverse QFT recovers the input;
- zero noise reproduces the ideal baseline;
- teleportation fidelity is 1 in ideal simulation;
- a Grover oracle marks the intended bit string;
- an ansatz at a special angle produces a known state.

Sanity tests should fail loudly when a convention or implementation is wrong.

8. Reproducibility

A reproducible project should record:

- Python version;
- package versions;
- installation commands;
- notebook or script entry point;
- random seeds if relevant;
- shot counts;
- backend information for hardware runs;
- any input parameters required.

A reader should not need to infer the environment from imports.

For a repository, a `requirements.txt` or `pyproject.toml` can make dependency setup explicit.

9. Separate theory, experiment, and interpretation

A clear notebook structure is:

Question

What are you testing?

Theory

What result does the model predict?

Method

What circuit and parameters are used?

Prediction

What should happen before execution?

Result

What happened?

Interpretation

Why does the result support or contradict the prediction?

Limitations

What does the experiment *not* demonstrate?

This structure is more important than decorative plotting.

10. Candidate capstone themes

Quantum random-number generator

Study measurement statistics of $|+\rangle$, sampling convergence, or hardware bias.

Bell-correlation explorer

Compare product and entangled states across measurement bases.

Teleportation demonstrator

Test a grid of Bloch-sphere states and report fidelity.

Grover puzzle solver

Encode a small marked-state problem and study iteration count.

Variational demonstrator

Minimize a one- or two-qubit Hamiltonian and compare with exact diagonalization.

Noise-aware circuit comparison

Compare logically equivalent circuits under a fixed noise model.

Superdense-coding demo

Encode/decode all four messages and quantify success under noise.

Mini algorithm library

Implement reusable tested functions for selected circuits with documentation.

11. Avoid unjustified “quantum advantage” claims

A circuit working on a simulator does not demonstrate quantum advantage.

A defensible quantum-advantage claim would require an appropriate comparison against strong classical methods, realistic resource accounting, and a problem scale where the quantum method achieves a relevant advantage.

For beginner capstones, prefer statements such as:

- “demonstrates the protocol”;
- “reproduces the predicted interference pattern”;
- “compares ideal and noisy performance”;
- “illustrates amplitude amplification.”

Precision is more credible than exaggerated claims.

12. Quantitative comparison

When comparing two conditions A and B , define a metric before running.

Examples:

$$\Delta p = p_A - p_B,$$

$$\text{relative error} = \frac{|x_{\text{obs}} - x_{\text{ref}}|}{|x_{\text{ref}}|},$$

$$\text{fidelity} = |\langle \psi_{\text{target}} | \psi_{\text{out}} \rangle|^2.$$

A plot without a defined metric can be visually suggestive but analytically weak.

13. Scope control

A capstone should be explainable in 5-8 minutes.

A useful scope test is:

- one main research question;
- one main quantum concept;
- one main metric;
- one baseline;
- one optional extension.

If a project needs several unrelated algorithms, a web application, hardware integration, and a new theoretical derivation all at once, it is probably too large for the course capstone.

14. What is genuinely quantum?

Students should explicitly identify the quantum ingredient.

Possible answers include:

- coherent superposition and interference;
- entanglement;
- phase kickback;
- amplitude amplification;
- a quantum Fourier transform;
- expectation values of a parameterized quantum state.

Avoid vague answers such as “it uses Qiskit” or “it runs on a quantum simulator.”

15. Week 15 deliverable

By the end of this week, the project should have:

- a one-sentence question;
- a written hypothesis;
- a primary metric;
- a baseline;
- a minimum working implementation;
- at least one sanity test;
- one baseline result;
- a short interpretation;
- a list of remaining issues for Week 16.

The project does not need to be polished yet. It needs to be *valid enough to test*.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>

- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.